QUESTIONS 2 / 2 completed

Sahred

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

PROGRAM EDITOR

Java

Run Save

```java

public class Main {
    public static void main(String[] args) throws InterruptedException {
        SharedResource resource = new SharedResource();

        MyThread thread1 = new MyThread(resource);
        MyThread thread2 = new MyThread(resource);

        thread1.start();
        thread2.start();

        thread1.join();
        thread2.join();

        System.out.println("Final Count: " + resource.getCount());
    }
}

class SharedResource {
    private int count = 0;

```

QUESTIONS 2 / 2 completed

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the

PROGRAM EDITOR

Java

Run Save

```java

import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws InterruptedException {

        AtomicInteger sharedValue = new AtomicInteger(0);

        int incrementsPerThread = 10000;

        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < incrementsPerThread; i++) {
                sharedValue.incrementAndGet();
            }
        });

        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < incrementsPerThread; i++) {
                sharedValue.incrementAndGet();
            }
        });
```

QUESTIONS 2 / 2 completed

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the

PROGRAM EDITOR

Java

Run Save

```java

import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws InterruptedException {

        AtomicInteger sharedValue = new AtomicInteger(0);

        int incrementsPerThread = 10000;

        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < incrementsPerThread; i++) {
                sharedValue.incrementAndGet();
            }
        });

        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < incrementsPerThread; i++) {
                sharedValue.incrementAndGet();
            }
        });
```